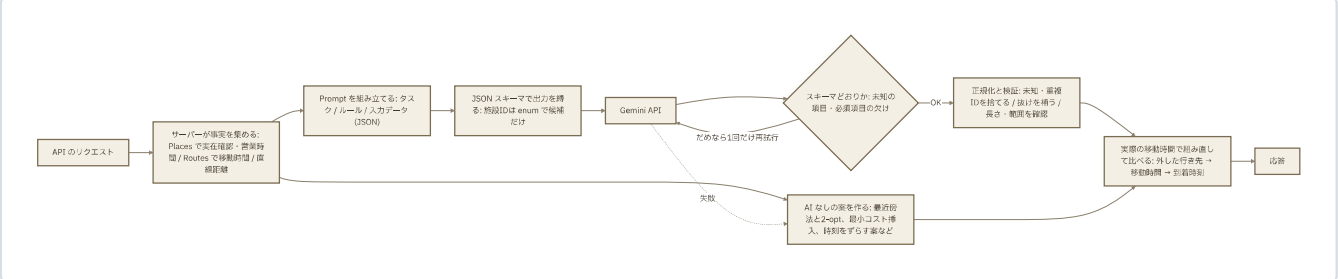
基本方針は「事実はサーバーが計算し、AI には選択・順番・文章だけを任せる」です。移動時間・営業時間・時刻は Maps の結果からサーバーが計算します。AI の出力は形と中身を確かめてから使い、AI が失敗しても返せる結果を用意しています。



API	AI に任せること	サーバー側の工夫	AI が失敗したと

② 自由文の解釈	1 回目：希望文を検索語に分ける。2 回目：実在の候補から最大 5 件を選び、理由をつける	Places で実在を確認した施設だけを enum で渡す	2 回目が失敗し ら、検索語ごと 頭の施設で返す
③ 旅程作成	施設・その日の営業時間・距離表を見て訪問順を決める	最近傍法＋2-opt の順番と両方組み、外した行き先 → 移動時間 → 到着時刻の順に比べる	基準の順番で返
⑥ 編集	固定の予定と変更点（追加・削除・優先度の変更）を見て、残りの順番を決める	変更のない施設は今の順番のまま、追加分だけを最小コストの位置に挿入した順番と比べる	基準の順番で返
⑦ 組み直し	近くの代替施設を使った案を最大 3 つ、短い説明（label）つきで作る	AI なしの案（時刻をずらす／優先度の低い予定を外す）と合わせて最大 5 案。結果が同じ案はまとめる	AI なしの案だ す。AI 案の組 に失敗したら、 案だけ飛ばす
⑧ 変更理由	サーバーが取り出した事実（時刻のずれ、延びた滞在、外れた理由、警告）を文章にする	理由の itemId は案の変更点だけ（enum）。種類（replace など）はサーバーの値を使う	`failed` を返す プリは定型文を し、確定はでき
⑩ 空き時間	滞在に使える時間に合う施設を選び、滞在時間と理由をつける。施設を決めない過ごし方も出す	前後の移動時間を Routes で求め、使える時間が 15 分未満の候補や営業時間外の候補を先に除く。最後に実際の時刻で間に合うか確かめる	502 を返す

データ

旅程（Itinerary）の形

③⑤⑥⑨⑪⑭⑯ が返す、アプリの中心データです。出発地（start）と終わる場所（end）は、施設なら `placeId`、地図上の地点なら `placeId: null` と `location`・`name` で表します。終わる場所への移動も最後の Leg（`toItemId: "end"`）として入ります。

```
{
  "id": "itn_8f2k", "version": 3, "title": "みなと町 日帰り旅", "date": "2026-11-14",
  "start": { "placeId": "ChIJ...", "name": "みなと町駅", "location": { "lat": 34.57, "
```

```
              "departAt": "2026-11-14T09:00:00+09:00" },
  "end":    { "placeId": null, "name": "駅前広場", "location": { "lat": 34.57, "lng": 1
              "arriveBy": "17:00", "arriveAt": "2026-11-14T16:05:00+09:00" },
  "items": [ { "itemId": "it_1", "placeId": "ChIJ...", "name": "朝市通り", "priority":
               "startAt": "...", "endAt": "...", "price": { "amount": null, "kind": "
               "openingHours": { "opensAt": "06:00", "closesAt": "12:00" }, "status":
  "legs":   [ { "fromItemId": "start", "toItemId": "it_1", "mode": "walk", "durationMi
              { "fromItemId": "it_1",  "toItemId": "end",  "mode": "bus",  "durationMi
  "summary": { "moveMin": 62, "priceTotal": { "amount": 0, "hasEstimate": true }, "er
  "dropped": [ { "placeId": "ChIJ...", "name": "海の美術館", "reason": "営業時間や終わる場
  "expiresAt": "2026-12-14T00:00:00+09:00"
}
```


Firestore コレクション	中身	自動削（TTL
`itineraries`	旅程本体（ownerUid、date、updatedAt、data）。1 人 20 件まで。一覧は ownerUid ↑ / date ↓ / __name__ ↓ の複合インデックスを使う	旅行日 30 日
`replans`	⑦ の組み直し案と、⑧ の理由文（案ごとの状態と処理権）	15 分
`rateLimits`	利用者ごとの 1 分あたりの呼び出し回数	2 分

アプリから Firestore には直接アクセスさせません（`firestore.rules` ですべて拒否）。TTL の実際の削除は遅れることがあるので、API 側でも期限を確かめます。

画面ごとの担当 API

画面名は、仕様の説明と Flutter 側（`front` ブランチ）の画面構成から推定したものです。

画面・場面	機能ID	使う API	この画面での役割
ホーム（旅程一覧）	F-08	④	旅行日の新しい順に 20 件ずつ表示。日」「これから」「終わった旅」への振けは、端末の日付でアプリが行う
旅程の管理	F-09 F-11	⑭ ⑮ ⑯	名前の変更、削除、別の日へのコピー
行き先の			施設名の入力補完と、「静かな市に行

行き先の入力・検索	F-01	① ②	施設名の入力補完と、「静かな寺に行い」のような自由文からの候補
地図から選ぶ	F-10	⑫ → ⑬	地図の範囲の施設（最大 20 件）と、だ施設のその日の営業時間・料金の目
旅程の作成	F-02	③	出発地・出発時刻、終わる場所・着き時刻、行き先から 1 日の旅程を作る
当日の行程表	F-06	⑤	予定・移動・集計を表示し、ETag をておく
旅程の編集	F-01 F-02 F-09 F-10	⑥	行き先・優先度・出発と終わり・旅行変えて作り直す
遅延時の組み直し	F-03 F-04	⑦ → ⑨	最大 5 案（AI なし 2 案＋AI 3 案）をて 1 案を確定
変更理由の表示	F-05	⑧	全体の理由（text）と、変更ごとの理（reasons）。pending の間は 1 秒ご取り直す
空き時間の提案と追加	F-07	⑩ → ⑪	寄れる施設（suggestions）と、施設めない過ごし方（ideas）。選んだ施設入する

API と外部サービスの対応

認証が必要な API はすべて、最初に Firebase Auth と Firestore の `rateLimits` を使います。下の表はそれ以外の呼び出しです。

API	Places	Routes	Gemini	Firestore	補足
① autocomplete	●	–	–	–	Autocomplete
② interpret	●	–	●	–	Gemini 2 回 + Text Search
⑫ nearby	●	–	–	–	Nearby Search（q があれば Tex Search）

⑬ 施設の詳細	●	–	–	–	Details（臨時営業・料金の目安む）
③ 旅程作成	●	●	●	●	2 つの順番を組むので Routes は で約 2 倍
④ 旅程一覧	–	–	–	●	複合インデックス
⑤ 旅程取得	–	–	–	●	
⑥ 編集	●	●	●	●	
⑭ 名前の変更	–	–	–	●	
⑮ 削除	–	–	–	●	組み直し案も削除
⑯ コピー	●	●	–	●	元の訪問順を使う
⑦ 組み直し	●	●	●	●	時刻調整だけで済むときは AI な
⑧ 変更理由	–	–	●	●	
⑨ 確定	–	–	–	●	
⑩ 空き時間	●	●	●	●	候補 8 件まで前後の移動を調べ
⑪ 予定の追加	●	●	–	●	
⓪ healthz	–	–	–	–	認証なし

各 API の機能

エラーは共通の形 `{"error": {"code", "message", "details"}}` で返します。主なエラーは 400 / 401 / 404 / 409 / 410 / 422 / 429 / 502 です。紫の欄は AI の役割です。

施設検索・地図 行き先・出発地・終わる場所を決める入口

① GET

`/places/autocomplete`

F-01 Places

施設名を入力している途中で候補を出す。

入力　`q`（1〜100 文字）、`lat/lng`（任意）

出力　`candidates[]`: placeId, name, address

② POST

/places/interpret

F-01　Gemini　Places

自由文の希望を、実在する施設の候補（最大 5 件）にする。

入力　`text`（1〜200 文字）、`near`（lat/lng か placeId、任意）

出力　`candidates[]`: place（placeId, name, address, location）, reason

AI 検索語への分解と、実在の候補からの選択の 2 段階。

⑫ GET

/places/nearby

F-10　Places　改訂で追加

地図に表示している範囲の施設を返す。

入力　`lat/lng`（必須）、`radius`（100〜5000m、既定 1000）、`q`（任意）

出力　`places[]`（最大 20 件）: placeId, name, address, location, typeLabel

⑬ GET

/places/:placeId

F-10　Places　改訂で追加

地図で選んだ施設の詳細と、その日の営業時間を返す。

入力　`date`（任意、既定は今日）

出力　place, typeLabel, openingHours（休みなら null）, price

判定　7 日以内は臨時営業・臨時休業を反映。前日から続く深夜営業は 00:00 からとして含める

旅程の作成・閲覧・管理　アプリの中心データ Itinerary を扱う

③ POST

/itineraries

F-02　Gemini　Places　Routes　Firestore

F-02 Gemini Places Routes Firestore

行き先と優先度から 1 日分の旅程を作り、保存する（201）。

入力　title（1〜100 文字）, date, start（placeId か location+name, departAt）, end（placeId か location+name, arriveBy）, places 1〜10 件（placeId, priority, stayMin 10〜480）

エラー　422 `MUST_ITEMS_INFEASIBLE`（必須が入らない）、`END_INFEASIBLE`（終わる場所に間に合わない）、`ITINERARY_LIMIT`（20 件）

AI 訪問順の提案。基準の順番と比べて良い方を採る。

④ GET

/me/itineraries

F-08 Firestore

自分の旅程を旅行日の新しい順に返す。

入力　`limit`（1〜50、既定 20）、`cursor`

出力　`itineraries[]`: id, title, date, start {name, departAt}, end {name, arriveAt}, placeCount, placeNames（3 件まで）, updatedAt, expiresAt と `nextCursor`

⑤ GET

/itineraries/:id

F-06 Firestore

旅程を 1 件返す。ETag に version が入る。

エラー　404（存在しない・他人のもの・期限切れ）

⑥ PUT

/itineraries/:id/places

F-01 F-02 F-09 F-10 Gemini

行き先・優先度・出発と終わり・旅行日を変えて作り直す。

入力　`If-Match`、places（全件）、date / start / end（任意、省略で今の値）

動き　完了済み・滞在中の予定は固定（遅延で延びた滞在のまま送り直せる）。旅行日を変えると全体を作り直す

AI 変更点だけを入れ替える部分的な組み直し。

⑭ PATCH

PATCH

/itineraries/:id

F-09 Firestore 改訂で追加

旅程の名前を変える（1〜100 文字、If-Match 必須）。

⑮ DELETE

/itineraries/:id

F-09 Firestore 改訂で追加

旅程と組み直し案を削除する（204、If-Match 必須）。

⑯ POST

/itineraries/:id/copy

F-11 Places Routes 改訂で追加

旅程を別の日にコピーする（201）。新しい日付の営業時間と時刻表で組み直す。

入力 date（必須）、title（任意）

遅延時の組み直し 当日の遅れに合わせて案を出し、選んだ案を反映する

⑦ POST

/itineraries/:id/replans

F-03 F-04 Gemini

遅延の影響を判定し、必要なら最大 5 案を返す。

入力 currentItemId と、delayMin（0〜600）か now

出力 replanId, basedOnVersion, expiresAt（15 分）, warnings（must_at_risk / end_at_risk / miss_transit）, options[]（source, label, changes, summaryBefore/After, preview）

AI 近くの代替施設を使う案（最大 3）。AI なしの 2 案と合わせて返す。

⑧ GET

/itineraries/:id/replans/:replanId/explanation

F-05 Gemini

案ごとの変更理由を返す。

入力 optionId

出力　status（pending / ready / failed）, text, reasons[]（itemId, type, reason）

AI サーバーが取り出した事実を、要約と変更ごとの理由にする。

⑨ POST

/itineraries/:id/replans/:replanId/accept

F-04 Firestore

選んだ案を旅程に反映する。If-Match は basedOnVersion。期限切れは 410。

空き時間の提案　予定の隙間に寄り道を足す

⑩ GET

/itineraries/:id/suggestions

F-07 Gemini

前後の移動を除いても十分に過ごせる近くの施設と、時間の過ごし方を返す。

入力　from, to（必須）、afterItemId（任意）

出力　suggestions[]（place, typeLabel, reason, stayMin, price, insertAfterItemId, legFromPrev, legToNext）、ideas[]（title, description, durationMin）

AI 候補の選択と滞在時間・理由、施設を決めない過ごし方。

⑪ POST

/itineraries/:id/items

F-07 Places Routes

施設を予定の間に挿入する（insertAfterItemId を省略すると末尾）。終わる場所に間に合わなければ 422。

図 4

ここまでの経緯

ブランチと PR の流れです。PR 11・12・14 は、それぞれ自己レビューとチームのレビュー指摘への対応を済ませてからマージしました。